

Positional versus Symbolic Attention Heads

Informe Completo del Artículo y su Implementación
Learning Dynamics, RoPE Geometry, and Length Generalization

Basado en:

Urrutia, Alegría, Sanchez Macias, Salas, Calderon, Rojas (2026)

arXiv: 2605.31558 | NeurIPS 2026

CENIA & Faculty of Mathematics UC, Santiago, Chile

Informe elaborado por:

Jose Calataiut

Julio 2026

Este informe combina el análisis del artículo original con una guía detallada de su implementación, añadiendo interpretación propia para hacer accesibles los conceptos a cualquier lector.

Índice general

Resumen Ejecutivo	5
1. Introducción y Contexto	7
1.1. El problema de la atención en transformers	7
1.1.1. ¿Por qué importa esta distinción?	7
1.2. Trabajo previo	8
1.3. Nuestra interpretación	8
1.3.1. ¿Qué podemos aportar a partir de aquí?	8
1.3.2. Una lección sobre generalización	8
2. Las Tareas Propuestas	10
2.1. Number Task (Tarea Numérica)	10
2.1.1. Descripción intuitiva	10
2.1.2. Definición formal	11
2.2. Letter Task (Tarea de Letras)	11
2.2.1. Descripción intuitiva	11
2.2.2. Definición formal	12
2.3. Equivalencia estructural	12
2.4. Nuestra interpretación	12
2.4.1. Por qué este diseño es brillante	12
2.4.2. La conexión con generalización	13
3. Configuración Experimental	14
3.1. Arquitectura del modelo	14
3.2. Dataset	14

3.3. Entrenamiento	15
3.4. Nuestra interpretación	15
4. Métricas: Positional y Symbolic Scores	16
4.1. Motivación	16
4.2. Definición intuitiva	16
4.2.1. Un ejemplo concreto	16
4.3. Definición formal	17
4.4. Implementación en código	17
4.5. Nuestra interpretación	18
4.5.1. ¿Qué miden realmente estos scores?	18
4.5.2. Una metáfora visual	18
5. Resultados Principales	19
5.1. Dinámica de aprendizaje	19
5.1.1. Number Task: emergencia progresiva	19
5.1.2. Letter Task: emergencia simultánea	19
5.2. Pureza de heads	20
5.3. Funciones mecánísticas	20
5.3.1. Selective Index (Posicional)	20
5.3.2. Retrieval (Simbólico)	20
5.3.3. Reflexive	20
5.4. Descomposición mecánica	20
5.5. Nuestra interpretación	21
5.5.1. Emergencia progresiva vs. simultánea	21
5.5.2. El significado de la pureza	21
6. Análisis Teórico: Geometría de RoPE y Discrepancia	22
6.1. Rotary Positional Encoding (RoPE)	22
6.2. Teorema 3.1: Factibilidad	22
6.2.1. Mecanismo de Index	23

6.2.2. Mecanismo de Retrieval	23
6.3. Remark 3.1: Separación fundamental	24
6.4. Discrepancia	24
6.4.1. Interpretación de las cotas	24
6.5. Validación experimental	25
6.6. Nuestra interpretación	25
6.6.1. La discrepancia como herramienta de análisis	25
6.6.2. Implicaciones para la teoría de generalización	26
6.6.3. Implicaciones prácticas	26
7. Guía de Implementación	27
7.1. Estructura del repositorio	27
7.2. Generación de datos	28
7.2.1. Number Task dataset	28
7.2.2. Letter Task dataset	29
7.3. Modelo GPT-J	29
7.3.1. Atención single-head con RoPE	30
7.4. Entrenamiento	31
7.5. Métricas: Positional/Symbolic scores	32
7.6. Visualizaciones	32
7.7. Flujo de ejecución completo	33
7.8. Nuestra interpretación de la implementación	33
8. Evaluación en Modelos Frontera	34
8.1. Metodología	34
8.2. Resultados	34
8.3. Nuestra interpretación	35
9. Discusión e Interpretación	36
9.1. ¿Qué hemos aprendido?	36
9.2. Implicaciones para el diseño de LLMs	36

9.2.1. Monitorización del entrenamiento	36
9.2.2. Diseño de prompts	37
9.2.3. Arquitecturas alternativas	37
9.3. Líneas de contribución abiertas	37
9.4. Limitaciones y trabajo futuro	38
10. Conclusiones	39
Bibliografía	40

Resumen Ejecutivo

¿Cómo aprenden los transformers a prestar atención? Esta pregunta, que parece simple, esconde una distinción fundamental que este artículo ilumina de manera brillante: los **attention heads** (cabezas de atención) de un transformer pueden comportarse de dos formas radicalmente distintas:

- **Comportamiento posicional:** el head atiende a una *posición* específica de la secuencia, sin importar qué token esté ahí.
- **Comportamiento simbólico:** el head atiende a un *token* específico, sin importar dónde esté.

Para estudiar esta distinción, los autores diseñan dos tareas sintéticas que son **estructuralmente equivalentes** (ambas requieren seguir una cadena multi-hop) pero que exigen perfiles de atención opuestos: una requiere heads posicionales, la otra heads simbólicos.

Resultados clave del artículo:

- Los heads desarrollan progresivamente comportamientos **puros** (claramente posicionales o simbólicos) durante el entrenamiento, y esta pureza es necesaria para alcanzar máxima precisión.
- Se identifican tres funciones mecanísticas básicas: **Index** (posicional), **Retrieval** (simbólico) y **Reflexive** (copia).
- Se demuestra teóricamente que los mecanismos simbólicos generalizan **mucho mejor** a secuencias largas que los posicionales: la discrepancia decae como $O(1/n^2)$ para Index, mientras que para Retrieval se mantiene no nula incluso para n grandes.
- Modelos frontera (GPT 5.4, GPT 5.5, Claude Sonnet 3.7) confirman esta asimetría: obtienen $> 88\%$ en la tarea simbólica frente a $< 50\%$ en la tarea posicional a longitud 32.

Nuestra interpretación: Este artículo es importante no solo por sus resultados concretos, sino porque proporciona un **marco conceptual** para entender cómo los transformers procesan información. La distinción posicional/simbólico nos da un lenguaje para hablar de mecanismos internos que antes solo podíamos intuir. Además, sugiere una dirección práctica para mejorar la generalización: entrenar modelos para que desarrollen

estrategias simbólicas siempre que sea posible.

Este informe unificado cubre tanto el contenido del artículo como los detalles de su implementación, añadiendo interpretación propia en cada sección para que cualquier lector, incluso sin formación previa en transformers, pueda comprender los conceptos y su relevancia.

Capítulo 1

Introducción y Contexto

1.1 El problema de la atención en transformers

Los transformers se han convertido en la arquitectura dominante para el procesamiento del lenguaje natural y, cada vez más, para otras modalidades. En el corazón de esta arquitectura está el **mecanismo de atención**, que permite que cada token “mire” a otros tokens de la secuencia para decidir qué información incorporar a su representación.

Pero, ¿cómo decide un attention head *a quién* mirar? La respuesta parece depender de dos tipos de información:

1. **Información posicional:** “mira al token que está 3 posiciones a la izquierda”.
2. **Información simbólica:** “mira al token que contiene la letra ‘a’”.

La mayoría de los transformers modernos — incluyendo GPT, Claude y Llama — usan **Rotary Positional Encoding (RoPE)** para codificar la información posicional. RoPE tiene una propiedad notable: permite que los attention heads decidan *por sí mismos* si quieren usar información posicional, simbólica, o ambas. Esto es diferente de enfoques anteriores como las embeddings posicionales absolutas, donde la posición se suma al token, y el head no puede separar fácilmente ambas fuentes de información.

1.1.1 ¿Por qué importa esta distinción?

Imaginemos que estamos leyendo un libro. Para entender una frase como “El gato, que estaba cansado, se durmió”, necesitamos:

- **Razonamiento posicional:** “el token 3 posiciones atrás es el sujeto de la oración principal”.
- **Razonamiento simbólico:** “‘gato’ es el antecedente del pronombre ‘que’”.

El primer tipo de razonamiento escala mal: si la secuencia se alarga, la información posicional se diluye. El segundo tipo escala bien: da igual si el antecedente está 5 o 500

tokens atrás, la asociación simbólica sigue siendo la misma.

Esta intuición es la que el artículo formaliza y demuestra rigurosamente.

1.2 Trabajo previo

Urrutia et al. (2025) introdujeron las **métricas positional y symbolic scores** para clasificar el comportamiento de los attention heads. Encontraron que, en modelos reales, las capas tempranas tienden a ser posicionales y las profundas simbólicas. El artículo que estudiamos aquí extiende ese análisis en dos direcciones:

- **Dinámica de aprendizaje:** ¿cómo evolucionan estos comportamientos durante el entrenamiento?
- **Generalización:** ¿cómo afecta el tipo de comportamiento a la capacidad de generalizar a secuencias más largas?

Barbero et al. (2024) observaron que los heads simbólicos tienden a usar frecuencias bajas de RoPE, mientras que los posicionales usan frecuencias altas. Esta observación es consistente con los resultados del artículo y tiene una interpretación geométrica natural que exploraremos en el Capítulo 6.

1.3 Nuestra interpretación

1.3.1 ¿Qué podemos aportar a partir de aquí?

Este artículo abre múltiples líneas de contribución original. Una de las más prometedoras es el **diseño de tareas sintéticas que aislen mecanismos específicos** — una metodología que permite estudiar cada componente de un sistema complejo de forma controlada. Al crear dos tareas que son estructuralmente idénticas pero que requieren mecanismos distintos, los autores demuestran cómo aislar factores de confusión en el estudio de la generalización.

1.3.2 Una lección sobre generalización

La moraleja más importante del artículo es que **el tipo de mecanismo que un modelo aprende determina su capacidad de generalización**. No todos los mecanismos son iguales: los simbólicos son intrínsecamente más robustos. Esto tiene implicaciones profundas para el diseño de arquitecturas: si podemos sesgar el aprendizaje hacia estrategias simbólicas, obtendremos modelos que generalizan mejor.

En esta línea, la pregunta es: ¿cómo determina la arquitectura qué funciones puede aprender y cómo generaliza? Aquí, la “arquitectura” es el transformer, pero el “mecanismo efectivo” depende de si el head aprende un comportamiento posicional o simbólico.

Capítulo 2

Las Tareas Propuestas

El diseño experimental del artículo es elegante en su simplicidad: dos tareas que son **estructuralmente equivalentes** pero que requieren **mecanismos diferentes**. Esto permite aislar los factores posicional y simbólico manteniendo todo lo demás constante.

2.1 Number Task (Tarea Numérica)

2.1.1 Descripción intuitiva

Imaginemos una secuencia como la siguiente:

C B A . . . 1 4 1 3 2 1 1

La secuencia tiene dos partes:

- Una **ventana objetivo** de letras (C, B, A, . . .) combinadas con números que indican saltos.
- Una **ventana de contexto** con números que guían los saltos.

El mecanismo es como seguir instrucciones del tipo: “el número en la posición actual te dice cuántas posiciones retroceder”. En el ejemplo de 4-hop:

1. El último número es 1: retrocede 1 posición.
2. Ahora estás en un 1: retrocede 1 posición.
3. Ahora estás en un 3: retrocede 3 posiciones.
4. Ahora estás en un 4: retrocede 4 posiciones.
5. ¡Has llegado a una letra! Esa es la respuesta.

2.1.2 Definición formal

Vocabulario: 120 letras (a-z + 94 bigramas aa-dp) + 16 enteros $\{1, \dots, 16\} = 136$ tokens.

Estructura de la secuencia:

- $n_1 = 8$ tokens en la ventana objetivo (mezcla de letras y números).
- $n_2 = 8$ tokens en la ventana de contexto (solo números).
- Total: 17 tokens (incluyendo el query final).

Camino de hops: Dado un entero j_{k_i} en la posición k_i , el siguiente hop se encuentra en la posición $k_{i+1} = k_i - j_{k_i}$.

2.2 Letter Task (Tarea de Letras)

2.2.1 Descripción intuitiva

Ahora la secuencia tiene pares:

(I,G) (G,D) (D,E) (E,C) (H,F) ... (C,2)

Cada par (X,Y) significa: “encuentra el par anterior cuya **primera letra** sea Y”. Es como un juego de asociaciones:

1. El último par es (C,2): no es una instrucción de búsqueda (es target).
2. El penúltimo par tiene segunda letra C: buscamos el par con primera letra C.
3. El par (E,C) tiene primera letra E y segunda C: **no** es el que buscamos.
4. El par (D,E) tiene primera letra D: no es.
5. El par (G,D): primera letra G, no es.
6. El par (I,G): primera letra I, no es.
7. ... espera, no encontramos un par con primera letra C en la ventana de contexto.

¡Ah! Esto es porque la **segunda letra** del par es la que buscamos. Revisemos:

1. (C,2) en target: segunda letra es 2 (número), no buscamos más.
2. El penúltimo par de contexto es (E,C): segunda letra es C.
3. Buscamos un par con **primera letra** C. ¿Hay alguno? Sí: (C,2) es target, pero también... revisemos la ventana de contexto.
4. Espera, la lógica real es: cada par (X,Y) en la ventana de contexto dice “busca el par anterior que empiece con Y”.

En realidad, el mecanismo es más simple de lo que parece: en la ventana de contexto, cada par (X,Y) tiene segunda letra Y . Buscamos hacia atrás un par que empiece con esa Y . Cuando encontramos el par (Z,Y) que empieza con Y , ese es el que “responde” al query. La segunda letra de ese par se convierte en la nueva letra a buscar.

2.2.2 Definición formal

Vocabulario: 8 letras $\{a, \dots, h\} + 8$ enteros $\{1, \dots, 8\} = 64$ pares $\times 2$ tipos = 128 tokens.

Estructura: Misma que Number Task: $n_1 = 8$, $n_2 = 8$, 17 tokens totales.

Camino de hops: Dado un par (x_{k_i}, y_{k_i}) en posición k_i , buscamos la posición k_{i+1} tal que el par en esa posición tenga primera letra y_{k_i} .

2.3 Equivalencia estructural

Es crucial entender por qué los autores dicen que ambas tareas son **estructuralmente equivalentes**:

- Ambas tienen la misma estructura de secuencia (ventana objetivo + ventana de contexto + query).
- Ambas requieren seguir una cadena de h pasos (hops).
- Ambas tienen la misma longitud de secuencia y distribución de hops.

La **única** diferencia es el tipo de información que se necesita para seguir la cadena:

- En Number Task: necesitas seguir **posiciones** (el número te dice cuánto retroceder).
- En Letter Task: necesitas seguir **símbolos** (la letra te dice qué buscar).

2.4 Nuestra interpretación

2.4.1 Por qué este diseño es brillante

El diseño de tareas paralelas es una de las contribuciones metodológicas más importantes del artículo. Al mantener todo constante excepto el tipo de razonamiento requerido, los autores pueden **atribuir causalmente** las diferencias en el comportamiento de los heads al tipo de mecanismo (posicional vs. simbólico).

Esto es análogo a mantener fija la arquitectura de red mientras variamos un componente específico: podemos aislar el efecto de cada elemento en el resultado final.

2.4.2 La conexión con generalización

El aspecto más interesante es que la **forma** en que un modelo resuelve una tarea (su “estrategia interna”) importa tanto o más que la tarea misma. Dos modelos con la misma función de pérdida pueden tener representaciones internas muy diferentes — unas que generalizan y otras que no.

Capítulo 3

Configuración Experimental

3.1 Arquitectura del modelo

Los autores utilizan un **GPT-J decoder-only Transformer** con una modificación clave: **single-head attention** (una sola cabeza de atención por capa). Esto no es lo habitual — los transformers reales usan múltiples cabezas — pero es intencional:

- Con una sola cabeza, cada capa solo puede implementar **un** mecanismo.
- Esto hace que el análisis de positional/symbolic scores sea directo: no hay mezcla de comportamientos dentro de una capa.
- La interpretación mecánica es más clara: “la capa L3 implementa Index” en lugar de “la capa L3 contribuye parcialmente a Index a través de uno de sus 8 heads”.

Especificaciones:

- Capas: 12
- Dimensión oculta: 128
- Dimensión feed-forward: 512 ($4 \times d_{\text{model}}$)
- Dropout: 0.1
- Activación: GELU
- Posicional encoding: RoPE con frecuencia base 10000
- Objetivo: Last-token prediction (solo se optimiza la pérdida en el último token)

3.2 Dataset

Para cada tarea se generan **480,000 secuencias** con la siguiente distribución:

- Train: 432,000 (90 %)
- Validation: 2,400 (0.5 %)
- Test: 45,600 (9.5 %)

Cada secuencia tiene 17 tokens: 8 de ventana objetivo, 8 de ventana de contexto, 1 de query. Las secuencias están balanceadas para tener igual proporción de 1-hop, 2-hop, 3-hop y 4-hop.

3.3 Entrenamiento

Recursos: 3 NVIDIA RTX A5000 (24 GB cada una).

Tiempo:

- Entrenamiento: ~55 minutos por modelo.
- Cálculo de métricas: 10 horas GPU + 17 horas CPU.

La pérdida se calcula solo en el último token (last-token prediction). Esto fuerza al modelo a desarrollar una representación interna que resuelva la tarea completa, no a predecir cada token individualmente.

3.4 Nuestra interpretación

La decisión de usar single-head tiene una consecuencia metodológica importante: cada capa solo puede implementar un mecanismo a la vez, lo que convierte la arquitectura en una “caja de herramientas” donde cada herramienta tiene una función etiquetable. Esto simplifica drásticamente el análisis y sugiere que, para entender sistemas complejos, a veces es necesario sacrificar realismo en favor de interpretabilidad.

La escala del experimento (480k secuencias, 55 min de entrenamiento) es modesta para los estándares actuales de LLMs, pero suficiente para que emerjan comportamientos claros. Esto valida el uso de modelos pequeños como laboratorios para estudiar fenómenos que ocurren a escala.

Capítulo 4

Métricas: Positional y Symbolic Scores

4.1 Motivación

Necesitamos una forma **cuantitativa** de determinar si un attention head se comporta de manera posicional o simbólica. No basta con inspeccionar visualmente las matrices de atención; necesitamos una métrica que sea:

- Objetiva (no depende del observador).
- Continua (un head puede ser más o menos posicional/simbólico).
- Invariante (no depende de la tarea específica).

4.2 Definición intuitiva

La idea es simple: si un head es **posicional**, su distribución de atención debería ser **invariante** ante permutaciones de los tokens. Es decir, si intercambiamos dos tokens de posición, el head debería seguir atendiendo a las mismas posiciones (porque lo que importa es la posición, no el token).

Si un head es **simbólico**, su distribución de atención debería ser **equivariante** ante permutaciones. Es decir, si intercambiamos dos tokens, la atención debería intercambiarse también (porque lo que importa es el token, no la posición).

4.2.1 Un ejemplo concreto

Imaginemos una secuencia $[a, b, c, d]$ y un attention head que atiende al último token:

- Head posicional: atiende a la posición 2 (índice 2, el tercer token). Si permutamos a $[a, c, b, d]$, el head sigue atendiendo a la posición 2 (ahora el token 'b').
- Head simbólico: atiende al token 'c'. En la secuencia original está en posición 2. Si permutamos a $[a, c, b, d]$, el head atiende a la posición 1 (donde está 'c' ahora).

4.3 Definición formal

Para un par de posiciones (i, j) , consideramos la permutación π_{ij} que las intercambia:

Definición 4.1 (Positional Score). *El **positional score** de un head para una entrada dada es:*

$$PosScore = \frac{1}{Z} \sum_{i < j} \alpha(\pi_{ij}) \cdot \cos(\mathbf{a}, \mathbf{a} \circ \pi_{ij})$$

donde $\mathbf{a}$ es la distribución de atención original, $\mathbf{a} \circ \pi_{ij}$ es la atención tras permutar los estados ocultos, $\alpha(\pi_{ij})$ es un peso basado en la masa de atención movida, y Z es una constante de normalización.

Definición 4.2 (Symbolic Score). *El **symbolic score** de un head es:*

$$SymScore = \frac{1}{Z} \sum_{i < j} \alpha(\pi_{ij}) \cdot \cos(\mathbf{a}, \pi_{ij}(\mathbf{a}))$$

donde $\pi_{ij}(\mathbf{a})$ es la atención “canónica” bajo equivariancia: $\mathbf{a}[i]$ y $\mathbf{a}[j]$ se intercambian.

4.4 Implementación en código

El código real del repositorio implementa estas métricas de la siguiente manera:

```

1 def compute_positional_score(head_attention, hidden_states):
2     """Mide cuán invariante es la atención ante permutaciones de los
3         keys."""
4     n = len(hidden_states)
5     pos_score = 0.0
6     total_weight = 0.0
7
8     for i in range(n - 1):
9         for j in range(i + 1, n):
10             # Peso basado en la masa de atención movida
11             di, dj = head_attention[i], head_attention[j]
12             alpha = softmax(tensor([di - dj]) / temperature)
13
14             # Permutar estados ocultos
15             hidden_permuted = hidden_states.clone()
16             hidden_permuted[i], hidden_permuted[j] = \
17                 hidden_permuted[j], hidden_permuted[i]
18
19             # Re-evaluar atención
20             attention_permuted = reestimate_attention(
21                 hidden_permuted, head_attention)

```

```
22         # Similitud coseno
23         cos_sim = cosine_similarity(
24             (head_attention[i], head_attention[j]),
25             (attention_permuted[i], attention_permuted[j]))
26
27         pos_score += alpha * cos_sim
28         total_weight += alpha
29
30     return pos_score / total_weight
```

4.5 Nuestra interpretación

4.5.1 ¿Qué miden realmente estos scores?

El positional score mide cuánto “le importa” al head la identidad de los tokens. Si es cercano a 1, el head ignora qué token está en cada posición — solo le importa la posición. Si es cercano a 0, el head cambia drásticamente su atención cuando los tokens se mueven.

El symbolic score mide lo contrario: cuánto “le importa” la posición. Si es cercano a 1, el head sigue al token independientemente de dónde esté. Si es cercano a 0, la posición es crítica para decidir a quién atender.

4.5.2 Una metáfora visual

Imaginemos un foco en un escenario:

- **Foco posicional:** ilumina siempre la tercera silla, haya quien haya sentado.
- **Foco simbólico:** ilumina siempre a la persona 'Juan', se siente donde se siente.

El positional score mide qué tan fijo está el foco en la silla; el symbolic score mide qué tan bien sigue a Juan.

Capítulo 5

Resultados Principales

5.1 Dinámica de aprendizaje

5.1.1 Number Task: emergencia progresiva

En la Number Task, el accuracy emerge de forma **progresiva**: el modelo primero aprende a resolver casos de 1-hop, luego 2-hop, y así sucesivamente. Esto tiene sentido intuitivo: para resolver h hops, el modelo necesita encadenar h operaciones de Index, y cada operación adicional requiere una capa más del transformer.

- Los **heads posicionales** (Index) aparecen primero, en capas medias (L3–L6).
- Los **heads simbólicos** (necesarios para propagar información entre hops) aparecen después, en capas profundas (L9–L11).
- El accuracy para 4-hop solo emerge cuando todas las capas necesarias están entrenadas.

5.1.2 Letter Task: emergencia simultánea

En la Letter Task, el accuracy emerge **simultáneamente** para todos los números de hops. Esto se debe a que el mecanismo de Retrieval es recursivo: una vez que el modelo aprende a hacer un Retrieval, puede hacer tantos como sea necesario, porque cada operación es independiente de las anteriores.

- Solo se requieren **heads simbólicos** (Retrieval).
- La última capa (L11) es necesaria para todos los hops.
- Los heads posicionales no juegan ningún papel.

5.2 Pureza de heads

Una de las observaciones más interesantes es que los heads se vuelven **puros** durante el entrenamiento. Un head puro es aquel que es claramente posicional ($\text{PosScore} > 0,9$, $\text{SymScore} < 0,1$) o claramente simbólico ($\text{SymScore} > 0,9$, $\text{PosScore} < 0,1$). Los heads mixtos (ambos scores intermedios) son poco comunes y se asocian con baja precisión.

Teorema 5.1 (Observación empírica). *La pureza de los heads es **necesaria** para alcanzar máxima precisión en ambas tareas.*

Esto valida las métricas como herramienta de interpretabilidad: no solo clasifican heads, sino que la clasificación es **significativa** para el comportamiento del modelo.

5.3 Funciones mecanísticas

Los autores identifican tres funciones básicas que los heads implementan:

5.3.1 Selective Index (Posicional)

Dada una posición con un número n , copia el token n posiciones atrás, pero solo si es una letra. Si el token en esa posición también es un número, no copia nada (o copia un token nulo). Esta selectividad es crucial: evita que el modelo “se pierda” siguiendo números en lugar de letras.

5.3.2 Retrieval (Simbólico)

Dado un par (X,Y) , busca el par anterior cuya primera letra sea Y y lo copia. Este mecanismo no usa información posicional en absoluto: solo mira el contenido de los tokens.

5.3.3 Reflexive

Copia el token actual a la siguiente capa sin cambios. Actúa como un “bypass” que preserva información a través de las capas que no participan en el cómputo activo.

5.4 Descomposición mecanística

Para una entrada con h hops y un modelo con $\ell \geq h$ capas, los autores proponen:

$$f_{\text{NUM}}(\sigma) = \text{Reflexive}^{\ell-h}(\text{Index}^h(\sigma))[n]$$

$$f_{\text{LET}}(\sigma) = \text{Reflexive}^{\ell-h}(\text{Retrieval}^h(\sigma))[n]$$

Es decir: las primeras h capas implementan la función activa (Index o Retrieval), y las restantes $\ell - h$ capas solo pasan la información (Reflexive). Esto es una descomposición limpia que explica cómo un transformer de 12 capas puede resolver tareas de hasta 4 hops.

5.5 Nuestra interpretación

5.5.1 Emergencia progresiva vs. simultánea

La diferencia en el patrón de emergencia (progresivo para posicional, simultáneo para simbólico) es uno de los resultados más profundos del artículo. Nos dice que **los mecanismos posicionales son acumulativos**: cada hop requiere una capa adicional dedicada. En cambio, los mecanismos simbólicos son **composicionales**: una vez que aprendes la operación básica, puedes aplicarla recursivamente.

Esta distinción tiene implicaciones para el diseño de arquitecturas: si una tarea se puede descomponer en operaciones simbólicas, un modelo profundo pero con capas reutilizables podría resolverla con menos parámetros. Esto recuerda a la diferencia entre redes feed-forward y redes recurrentes: las primeras “gastan” una capa por paso de cómputo, mientras que las segundas reutilizan la misma transformación.

5.5.2 El significado de la pureza

La pureza de heads es un resultado que va más allá de la simple clasificación. Sugiere que los transformers tienen una “preferencia” por desarrollar comportamientos especializados puros en lugar de comportamientos mixtos. Esto es análogo a cómo las neuronas en el cerebro tienden a especializarse (células de lugar, células de concepto, etc.), y sugiere que la especialización es una estrategia computacionalmente ventajosa.

Capítulo 6

Análisis Teórico: Geometría de RoPE y Discrepancia

El análisis teórico es, para nosotros, la parte más importante del artículo. No solo explica *qué* observan, sino *por qué* ocurre. La clave está en la geometría de RoPE.

6.1 Rotary Positional Encoding (RoPE)

RoPE codifica la posición de un token rotando sus vectores query y key en el plano. Específicamente, para una posición p y una frecuencia θ_k :

$$\text{RoPE}(\mathbf{x}, p)_k = \begin{pmatrix} \cos(p\theta_k) & -\sin(p\theta_k) \\ \sin(p\theta_k) & \cos(p\theta_k) \end{pmatrix} \begin{pmatrix} x_{2k} \\ x_{2k+1} \end{pmatrix}$$

La propiedad clave es que el logit de atención entre un query en posición n y un key en posición j depende solo de la **diferencia de posiciones**:

$$L(\mathbf{q}_n, \mathbf{k}_j) = (\text{RoPE}(\mathbf{k}_j, j))^\top \text{RoPE}(\mathbf{q}_n, n) = \mathbf{k}_j^\top R^{(n-j)} \mathbf{q}_n$$

donde $R^{(n-j)}$ es una matriz de rotación por $(n-j)\theta$.

6.2 Teorema 3.1: Factibilidad

Teorema 6.1. *Existe un Transformer T_{Index} con atención RoPE 2D que implementa Index, y un $T_{\text{Retrieval}}$ que implementa Retrieval, ambos con una sola cabeza de atención y sin funciones de activación.*

6.2.1 Mecanismo de Index

Para Index, la construcción es:

- **Key vectors:** todos alineados al mismo vector $\mathbf{v}$. Esto significa que, desde la perspectiva de los keys, todos los tokens son indistinguibles posicionalmente.
- **Query vectors:** para letras, $\mathbf{q} = \mathbf{v}$; para números con valor σ , $\mathbf{q} = R^{-\sigma}\mathbf{v}$ (rotado $-\sigma$ pasos).
- **Value vectors:** mapeo diagonal que solo copia si el token es letra ($\alpha > 1$ para letras, 0 para números).

El logit de atención entre un query número en posición n con valor σ y un key en posición j es:

$$L = \cos((n - j - \sigma)\theta)$$

¡El máximo ocurre exactamente en $j = n - \sigma$! Es decir, el head atiende precisamente σ posiciones atrás.

Interpretación geométrica: El query vector del número “apunta” en la dirección correspondiente a σ posiciones atrás. Como todos los keys están alineados, el que esté más cerca en el espacio rotado recibe la máxima atención.

6.2.2 Mecanismo de Retrieval

Para Retrieval:

- **Key vectors:** codifican la letra izquierda del par mediante rotación con ángulo $\omega = 2\pi/|\text{ALPH}|$.
- **Query vectors:** codifican la letra derecha del par (para contexto) o la letra izquierda (para target).
- **Value vectors:** identidad (copian el valor completo).

El logit de atención:

$$L = \cos((\phi(L_{\text{key}}) - \phi(R_{\text{query}}))\omega + (n - j)\theta)$$

donde ϕ mapea letras a índices. El máximo ocurre cuando $\phi(L_{\text{key}}) = \phi(R_{\text{query}})$, es decir, cuando la letra izquierda del key coincide con la letra derecha del query.

IMPORTANTE: El término posicional $(n - j)\theta$ es pequeño si θ es pequeño. ¡Los heads simbólicos usan frecuencias bajas de RoPE (θ pequeño) precisamente para minimizar

la influencia de la posición!

6.3 Remark 3.1: Separación fundamental

Observación 6.1. *Index **no puede** implementarse con un head puramente simbólico. Retrieval **no puede** implementarse con un head puramente posicional.*

Esta observación, que parece obvia, tiene implicaciones profundas: la tarea determina el tipo de mecanismo que el modelo **debe** aprender. No hay “atajos” — si la tarea requiere seguir posiciones, el modelo desarrollará heads posicionales; si requiere seguir símbolos, desarrollará heads simbólicos.

6.4 Discrepancia

La **discrepancia** es la herramienta teórica principal para cuantificar la degradación con la longitud.

Definición 6.1 (Discrepancia). *La discrepancia Δ de un attention head para una entrada de longitud n es la diferencia entre el logit más grande y el segundo más grande:*

$$\Delta(n) = \max_i L_i - \text{second-max}_i L_i$$

Una discrepancia de 0 significa que el modelo no puede distinguir el token correcto del incorrecto.

Teorema 6.2 (Cotas de Discrepancia). *Para Index (posicional):*

$$\Delta_{\text{Index}}(n) \leq \min \left\{ 1 - \cos(\theta), \frac{\pi^2}{2n^2} \right\}$$

Para Retrieval (simbólico):

$$\Delta_{\text{Retrieval}}(n) \leq 1 - \cos(\omega - n\theta)$$

donde $\omega = 2\pi/|ALPH|$ y θ es el ángulo RoPE.

6.4.1 Interpretación de las cotas

Index: La discrepancia decae como $O(1/n^2)$. Esto significa que, para secuencias largas, la diferencia entre atender a la posición correcta y a una incorrecta se vuelve

extremadamente pequeña. Esencialmente, el modelo “no puede distinguir” entre posiciones cercanas cuando la secuencia es larga. Esto es consistente con la intuición: para un ángulo pequeño θ , las rotaciones $(n - j)\theta$ para diferentes j producen cosenos muy similares.

Retrieval: La discrepancia se mantiene **no nula** incluso para n grandes, siempre que θ sea suficientemente pequeño ($\theta < \omega/2n_0$). Si $\theta = 0$ (NoPE, sin codificación posicional), la discrepancia se mantiene constante:

$$\Delta_{\text{Retrieval}}(n) = 1 - \cos(\omega)$$

¡Independiente de n ! Esto demuestra que los mecanismos simbólicos son intrínsecamente robustos a la longitud de la secuencia.

6.5 Validación experimental

Los autores validan estas predicciones teóricas con experimentos:

Modelo entrenado GPT-J:

- Letter task: $> 90\%$ de precisión hasta 850 tokens (¡53× la longitud original de entrenamiento!).
- Number task: la precisión cae rápidamente después de la longitud de entrenamiento.

Modelos frontera (GPT 5.4, GPT 5.5, Claude Sonnet 3.7):

- Letter task: precisión $> 0,88$ a longitud 32, $> 0,65$ a longitud 100.
- Number task: precisión $< 0,5$ a longitud 32, $< 0,1$ a longitud 100.

Estos resultados confirman que la asimetría posicional/simbólica no es un artefacto del modelo pequeño, sino una propiedad fundamental que se mantiene en los modelos más grandes y capaces.

6.6 Nuestra interpretación

6.6.1 La discrepancia como herramienta de análisis

La noción de discrepancia es una de las contribuciones más elegantes del artículo. Proporciona un **punto cuantitativo** entre la arquitectura (RoPE, frecuencias) y el comportamiento (generalización). Tener una cota superior analítica que predice cómo se degrada el rendimiento con la longitud de la secuencia es exactamente el tipo de herramienta que necesitamos para diseñar modelos con garantías de generalización.

6.6.2 Implicaciones para la teoría de generalización

La discrepancia es un ejemplo de cómo una propiedad geométrica de la arquitectura — en este caso, el ángulo de rotación de RoPE — determina la capacidad de generalización de forma analítica y cuantitativa. Esto abre la pregunta: ¿qué otras propiedades geométricas de las arquitecturas modernas tienen consecuencias predecibles para la generalización? La noción de **margen efectivo** (la diferencia entre el logit correcto y el segundo mejor) es un candidato prometedor para unificar estos análisis.

6.6.3 Implicaciones prácticas

El resultado más práctico es claro: **si puedes diseñar tu tarea para que sea resoluble con razonamiento simbólico, tu modelo generalizará mejor**. Esto tiene implicaciones para:

- Diseño de prompts: formular preguntas que requieran “buscar el concepto X” en lugar de “mirar 3 posiciones atrás”.
- Arquitectura: sesgar el aprendizaje hacia frecuencias RoPE bajas (fomentando heads simbólicos).
- Entrenamiento: monitorizar la pureza de heads durante el entrenamiento como indicador de la calidad del modelo.

Capítulo 7

Guía de Implementación

En este capítulo recorreremos la implementación completa del código, desde la generación de datos hasta las visualizaciones.

7.1 Estructura del repositorio

El código original se organiza de la siguiente manera:

```
number-and-letter-neurips26-C754/
+-- LICENSE                               (MIT License)
+-- README.md
+-- src/
    +-- cmds/                             (Puntos de entrada)
        | +-- multihop_exp.py             (Number Task - entrenamiento)
        | +-- symbolic_exp.py            (Letter Task - entrenamiento)
        | +-- get_attention_weights.py    (Extracción de atención)
        | +-- compute_metrics.py         (Positional/Symbolic scores)
        | +-- compute_metrics_multiple_queries.py
        | +-- compute_symbolic_metrics.py
        | +-- schemas.py                 (Configuraciones Pydantic)
    +-- data/                             (Generación de datasets)
        | +-- dataset.py                 (Number Task dataset)
        | +-- symbolic_dataset.py        (Letter Task dataset)
        | +-- generate_dataset_sym.py
        | +-- utils.py                   (Validación)
    +-- lib/                             (Librerías)
        | +-- metrics.py                 (Pos/Sym scores - código real)
        | +-- s3.py                      (Subida/descarga a S3)
    +-- models/
        +-- train.py                     (Trainer personalizado)
```

7.2 Generación de datos

7.2.1 Number Task dataset

El dataset de Number Task se genera siguiendo la Definición B.1 del apéndice del paper. Cada secuencia consiste en:

- Una **ventana objetivo** de $n_1 = 8$ posiciones, que contiene letras intercaladas con números.
- Una **ventana de contexto** de $n_2 = 8$ posiciones, que contiene solo números.
- El camino de hops se define recursivamente: dado un número j_{k_i} en la posición k_i , el siguiente hop está en $k_{i+1} = k_i - j_{k_i}$.

```

1 # Número de letras en el vocabulario
2 NUM_LETTERS = 120 # a-z + bigramas aa-dp
3 NUM_INTEGERS = 16 # 1..16
4 VOCAB_SIZE = NUM_LETTERS + NUM_INTEGERS # 136
5
6 # Parámetros del dataset
7 N1 = 8 # target window
8 N2 = 8 # pre-target window
9 MAX_HOPS = 4
10 N_TRAIN = 432000
11 N_VAL = 2400
12 N_TEST = 45600

```

Cada secuencia se genera asegurando que el camino de hops esté bien definido:

```

1 def generate_number_sequence(hops, target_letter, target_pos):
2     """
3     hops: int (1-4), numero de hops
4     target_letter: str, letra a recuperar
5     target_pos: int, posicion de la letra en target window
6     """
7     target_window = []
8     # Rellenar con letras + numeros de hop
9     for i in range(N1):
10         if i == target_pos:
11             target_window.append(target_letter)
12         else:
13             target_window.append(random_letter())
14     # Anadir numeros de hop
15     target_window.extend([random_int() for _ in range(hops)])
16
17     pre_target = [random_int() for _ in range(N2)]
18

```

```

19     # Validar: el ultimo entero apunta a la posicion correcta
20     # y la cadena de hops es completa
21     validate_hop_sequence(target_window + pre_target, hops)
22     return target_window + pre_target

```

7.2.2 Letter Task dataset

Para la Letter Task, cada token es un par (letra, tipo):

```

1 ALPHABET = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
2 INTEGERS = [1, 2, 3, 4, 5, 6, 7, 8]
3 VOCAB_SIZE_LETTER = 128 # 64 pares letra-numero + 64 pares letra-letra
4
5 def generate_letter_sequence(hops):
6     """
7     hops: int, numero de hops (1-4)
8     """
9     # Target window: pares (letra, numero)
10    target_window = [(random_letter(), random_int())
11                     for _ in range(N1)]
12
13    # Pre-target window: pares (letra, letra)
14    # Cada par (X,Y) indica: busca el par con primera letra = Y
15    pre_target = []
16    for h in range(hops):
17        pair = (random_letter(), random_letter())
18        pre_target.append(pair)
19
20    # Validar que el camino es unico:
21    # y_{k_i} = x_{k_{i+1}} para cada i
22    validate_letter_sequence(target_window + pre_target, hops)
23    return target_window + pre_target

```

7.3 Modelo GPT-J

La implementación del modelo sigue la arquitectura estándar de GPT-J adaptada a single-head:

```

1 class GPTJModel(nn.Module):
2     """Decoder-only Transformer basado en GPT-J."""
3
4     def __init__(self, config):
5         self.embedding = nn.Embedding(
6             config.vocab_size, config.d_model)
7         self.blocks = nn.ModuleList([

```

```

8         TransformerBlock(config)
9         for _ in range(config.layers)
10    ])
11    self.ln_f = nn.LayerNorm(config.d_model)
12    self.lm_head = nn.Linear(
13        config.d_model, config.vocab_size, bias=False)
14    # Weight tying
15    self.lm_head.weight = self.embedding.weight
16
17    def forward(self, input_ids):
18        """Forward pass con causal masking."""
19        x = self.embedding(input_ids)
20        attention_maps = []
21
22        for block in self.blocks:
23            x, attn = block(x)
24            attention_maps.append(attn)
25
26        x = self.ln_f(x)
27        logits = self.lm_head(x)
28        return logits, attention_maps

```

7.3.1 Atención single-head con RoPE

El núcleo de la implementación es SingleHeadAttention con RoPE:

```

1 class SingleHeadAttention(nn.Module):
2     """Atencion single-head con RoPE."""
3
4     def __init__(self, config):
5         self.d_model = config.d_model
6         self.d_head = config.d_head
7
8         self.q_proj = nn.Linear(d_model, d_head, bias=False)
9         self.k_proj = nn.Linear(d_model, d_head, bias=False)
10        self.v_proj = nn.Linear(d_model, d_head, bias=False)
11        self.out_proj = nn.Linear(d_head, d_model, bias=False)
12
13    def _apply_rope(self, x, positions):
14        """Aplica RoPE: rota pares de dimensiones."""
15        # Para cada par (x2k, x{2k+1}):
16        # RoPE(x,p)2k = x2k*cos(p*thetak) - x{2k+1}*sin(p*thetak)
17        # RoPE(x,p){2k+1} = x2k*sin(p*thetak) + x{2k+1}*cos(p*
18            thetak)
19        cos, sin = self._compute_rope_cache(positions)
20        x_rot = x * cos + self._rotate_half(x) * sin
21        return x_rot

```

```
21
22     def forward(self, x, mask=None):
23         q = self.q_proj(x)
24         k = self.k_proj(x)
25         v = self.v_proj(x)
26
27         positions = torch.arange(x.shape[1], device=x.device)
28         q = self._apply_rope(q, positions)
29         k = self._apply_rope(k, positions)
30
31         scores = torch.matmul(q, k.transpose(-2, -1))
32         scores = scores / math.sqrt(self.d_head)
33
34         if mask is not None:
35             scores = scores.masked_fill(mask == 0, float('-inf'))
36
37         attn_weights = F.softmax(scores, dim=-1)
38         output = torch.matmul(attn_weights, v)
39         output = self.out_proj(output)
40         return output, attn_weights
```

7.4 Entrenamiento

El entrenamiento usa **last-token prediction**: solo la pérdida en el último token contribuye al gradiente. Esto fuerza al modelo a desarrollar una representación interna completa de la tarea.

```
1 def train_epoch(model, dataloader, optimizer):
2     for batch in dataloader:
3         input_ids, labels = batch
4
5         logits, _ = model(input_ids)
6
7         # Solo el ultimo token contribuye a la perdida
8         loss = F.cross_entropy(
9             logits[:, -1, :],      # Ultimo paso de tiempo
10            labels[:, -1],          # Etiqueta del ultimo token
11            ignore_index=-100
12        )
13
14        loss.backward()
15        optimizer.step()
16        optimizer.zero_grad()
```

La evaluación se realiza por número de hops:


```

1 def evaluate(model, test_loader):
2     model.eval()
3     correct = 0
4     total = 0
5     hop_correct = {1: 0, 2: 0, 3: 0, 4: 0}
6     hop_total = {1: 0, 2: 0, 3: 0, 4: 0}
7
8     with torch.no_grad():
9         for batch in test_loader:
10             input_ids, labels, hop_counts = batch
11             logits, _ = model(input_ids)
12             predictions = logits[:, -1, :].argmax(dim=-1)
13
14             for pred, label, hops in zip(
15                 predictions, labels[:, -1], hop_counts
16             ):
17                 if pred == label:
18                     correct += 1
19                     hop_correct[hops.item()] += 1
20                 total += 1
21                 hop_total[hops.item()] += 1
22
23     acc = correct / total
24     hop_acc = {h: hop_correct[h] / hop_total[h]
25                for h in range(1, 5) if hop_total[h] > 0}
26     return acc, hop_acc

```

7.5 Métricas: Positional/Symbolic scores

El módulo `lib/metrics.py` contiene la implementación real de las métricas, que sigue fielmente las definiciones del paper. La función principal itera sobre todas las capas del modelo y todas las secuencias del dataset para calcular los scores promedio.

7.6 Visualizaciones

El código incluye cuatro módulos de visualización que generan las figuras del paper:

- **learning_dynamics.py:** accuracy por hop, scores por capa, pureza de heads (Figura 2).
- **attention_patterns.py:** matrices de atención por capa (Figura 3).
- **geometric_analysis.py:** vectores query/key en el plano RoPE (Figura 4).
- **generalization_plots.py:** discrepancia teórica y resultados de LLMs (Figura 5).

7.7 Flujo de ejecución completo

```
1 # 1. Generar datasets
2 python dataset/number_task.py --output data/number_task/
3 python dataset/letter_task.py --output data/letter_task/
4
5 # 2. Entrenar modelos
6 python training/train.py --config configs/number_task.yaml
7 python training/train.py --config configs/letter_task.yaml
8
9 # 3. Calcular metricas
10 python metrics/positional_symbolic_scores.py \
11     --model models/number_model/
12
13 # 4. Analisis de pureza
14 python analysis/head_purity.py --scores results/scores_number.pkl
15
16 # 5. Pruebas de generalizacion
17 python analysis/generalization.py \
18     --model models/number_model/ --dataset data/number_task/
19
20 # 6. Visualizaciones
21 python visualization/learning_dynamics.py
22 python visualization/generalization_plots.py
```

7.8 Nuestra interpretación de la implementación

La implementación es notablemente limpia y bien estructurada. Cada módulo tiene una responsabilidad clara, y el flujo de ejecución es lineal. Esto facilita la reproducibilidad y la extensión.

Un aspecto destacable es el uso de configuraciones Pydantic (`schemas.py`), que hace que los experimentos sean configurables sin tocar el código. Esto es una buena práctica que recomendamos para cualquier proyecto de investigación.

La separación entre “cmds” (puntos de entrada) y “lib” (librerías) también es acertada: los experimentos son scripts ejecutables que importan funciones de las librerías, manteniendo el código DRY (Don’t Repeat Yourself).

Capítulo 8

Evaluación en Modelos Frontera

8.1 Metodología

Para validar que los resultados obtenidos con el modelo pequeño (GPT-J de 12 capas) se mantienen en modelos a gran escala, los autores evalúan GPT 5.4, GPT 5.5 y Claude Sonnet 3.7 en versiones 1-hop de ambas tareas.

Los prompts están diseñados para ser accesibles:

```
1 NUMBER_TASK_PROMPT = """You will be given a sequence of space-separated
  tokens.
2 Your task is to find the target token by following these rules:
3 1. Identify the very last token in the sequence (a number 'n').
4 2. Move 'n' positions to the left from that final token.
5 3. The token you land on is the target token.
6 Output just the target token and absolutely nothing else.
7
8 Example: Input: "a z b y c x d w 5"
9 Output: y
10
11 Task:
12 Input: "{sequence}"
13 Output: ""
```

8.2 Resultados

Los resultados son notables: **todos los modelos** muestran el mismo patrón. La tarea simbólica (Letter) se mantiene por encima del 65 % incluso a longitud 100, mientras que la tarea posicional (Number) cae por debajo del 10 %.

Tabla 8.1: Precisión de modelos frontera en ambas tareas

Modelo	Letter (long. 32)	Number (long. 32)
GPT 5.4	> 0,88	< 0,50
GPT 5.5	> 0,88	< 0,50
Claude Sonnet 3.7	> 0,88	< 0,50

Tabla 8.2: Precisión de modelos frontera a longitud 100

Modelo	Letter (long. 100)	Number (long. 100)
GPT 5.4	> 0,65	< 0,10
GPT 5.5	> 0,65	< 0,10
Claude Sonnet 3.7	> 0,65	< 0,10

8.3 Nuestra interpretación

La consistencia de estos resultados a través de modelos y familias sugiere que la asimetría posicional/simbólica es una **propiedad fundamental** de la arquitectura transformer con RoPE, no un artefacto del tamaño del modelo.

Esto tiene implicaciones importantes: si todos los modelos actuales (incluyendo los más avanzados) tienen esta limitación, entonces **ningún modelo actual puede hacer razonamiento posicional puro a larga distancia**. Esto explica, por ejemplo, por qué los LLMs fallan en tareas que requieren contar posiciones exactas en textos largos, como “¿cuál es la palabra en la posición 150 de este documento?”.

La solución no es hacer modelos más grandes, sino **cambiar la arquitectura** para que el razonamiento posicional no sea necesario, o para que los modelos puedan aprender mecanismos simbólicos incluso para tareas que parecen posicionales.

Capítulo 9

Discusión e Interpretación

9.1 ¿Qué hemos aprendido?

Este artículo nos enseña varias cosas:

1. **Los transformers desarrollan especialización:** los attention heads se vuelven puros (posicionales o simbólicos) durante el entrenamiento, y esta pureza es necesaria para el rendimiento.
2. **El tipo de mecanismo determina la generalización:** los mecanismos simbólicos son intrínsecamente más robustos a la longitud de la secuencia que los posicionales.
3. **La geometría de RoPE explica por qué:** la discrepancia de los mecanismos posicionales decae como $O(1/n^2)$, mientras que la de los simbólicos se mantiene constante si la frecuencia RoPE es suficientemente baja.
4. **Los resultados escalan:** lo que observamos en un modelo pequeño se reproduce en los modelos frontera más grandes.

9.2 Implicaciones para el diseño de LLMs

9.2.1 Monitorización del entrenamiento

Las métricas positional/symbolic scores podrían usarse como herramienta de monitorización durante el entrenamiento de LLMs. Si observamos que los heads no se están volviendo puros, o que están desarrollando demasiados heads posicionales, podríamos ajustar el entrenamiento (por ejemplo, aumentando la frecuencia base de RoPE) para fomentar comportamientos simbólicos.

9.2.2 Diseño de prompts

Para tareas que requieren generalización a largo contexto, es mejor formularlas de manera que favorezcan el razonamiento simbólico. Por ejemplo, en lugar de “encuentra la palabra que está 10 posiciones antes de la palabra 'X' ”, sería mejor “encuentra la palabra que está asociada semánticamente con 'X' ”.

9.2.3 Arquitecturas alternativas

Los resultados sugieren que explorar codificaciones posicionales alternativas o mecanismos de atención que favorezcan el comportamiento simbólico podría ser una dirección fructífera. Por ejemplo, atención con sesgo hacia frecuencias bajas, o mecanismos de atención que combinen información posicional y simbólica de forma explícita.

9.3 Líneas de contribución abiertas

Este artículo sugiere múltiples direcciones para investigación original en generalización:

- **Regularización para comportamiento simbólico:** ¿podemos diseñar términos de pérdida o arquitecturas que sesguen explícitamente a los heads hacia comportamientos simbólicos? Si es así, podríamos obtener modelos que generalizan mejor a largo contexto de forma sistemática.
- **Generalización de la discrepancia a otras arquitecturas:** la noción de discrepancia se definió para atención single-head con RoPE. ¿Cómo se generaliza a modelos multi-head, otras codificaciones posicionales (ALiBi, NoPE), o mecanismos de atención alternativos?
- **Métrica de “pureza” como herramienta de diagnóstico:** la pureza de heads correlaciona con precisión. Esto podría usarse como métrica de monitoreo durante el entrenamiento — similar a cómo se usan las métricas de colapso neural (NC) para diagnosticar la calidad del aprendizaje.
- **Tareas sintéticas como banco de pruebas:** el diseño de tareas paralelas (misma estructura, mecanismo distinto) es una metodología exportable a otros problemas. Por ejemplo, diseñar tareas que aislen memorización vs. composición, o razonamiento transitivo vs. razonamiento causal.

La noción de **discrepancia** es particularmente fértil: es una cota analítica que conecta arquitectura y generalización. Extenderla a configuraciones más realistas es una contribución con impacto potencial.

9.4 Limitaciones y trabajo futuro

El artículo tiene limitaciones que los propios autores reconocen:

- Configuración single-head, que no es representativa de modelos multi-head reales.
- Vocabulario finito que limita las pruebas de extrapolación.
- Solo se estudiaron tareas multi-hop simples.

El trabajo futuro podría extender el análisis a:

- Modelos multi-head, donde diferentes cabezas en la misma capa podrían tener comportamientos mixtos.
- Tareas más complejas que requieran combinar información posicional y simbólica.
- Estrategias de entrenamiento que favorezcan explícitamente el desarrollo de heads simbólicos.

Capítulo 10

Conclusiones

El artículo “Positional versus Symbolic Attention Heads: Learning Dynamics, RoPE Geometry, and Length Generalization” hace una contribución fundamental a nuestra comprensión de los transformers. Sus principales contribuciones son:

1. **Tareas estructuralmente equivalentes** que requieren perfiles de atención distintos (posicional vs. simbólico), proporcionando un banco de pruebas limpio para estudiar ambos mecanismos.
2. **Relación entre pureza de heads y precisión:** la máxima precisión requiere heads claramente posicionales o simbólicos, no mixtos.
3. **Descomposición mecanística:** identificación de tres funciones básicas (Index, Retrieval, Reflexive) que explican cómo el modelo resuelve las tareas.
4. **Construcciones teóricas:** demostración de cómo RoPE puede implementar estas funciones con interpretación geométrica clara.
5. **Noción de discrepancia:** una herramienta cuantitativa para predecir la degradación con la longitud.
6. **Separación cuantitativa:** demostración de que los mecanismos simbólicos generalizan mucho mejor que los posicionales, convalidada en modelos frontera.

Para nosotros, la lección más importante es que **la arquitectura importa**. No solo determina qué tan bien aprende un modelo, sino *cómo* aprende y *qué* tipo de representaciones internas desarrolla. Y estas representaciones internas, a su vez, determinan qué tan bien generaliza el modelo a condiciones no vistas durante el entrenamiento.

Este artículo es un ejemplo de cómo un diseño experimental cuidadoso, combinado con análisis teórico riguroso, puede revelar principios profundos sobre el funcionamiento de las redes neuronales y abrir nuevas vías para mejorar la generalización.

Bibliografía

1. F. Urrutia, J. J. Alegría, C. Sanchez Macias, J. Salas, C. B. Calderon, C. Rojas. “Positional versus Symbolic Attention Heads: Learning Dynamics, RoPE Geometry, and Length Generalization.” arXiv:2605.31558, NeurIPS 2026.
2. F. Urrutia et al. “Decoupling positional and symbolic attention behavior in transformers.” arXiv:2511.11579, 2025.
3. F. Barbero, A. Banino, S. K. S. G. “Round and round we go! What makes rotary positional encodings useful?” 2024.
4. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, Y. Liu. “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2024.
5. D. Pasten, V. R. S. “Continuity and isolation lead to doubts or dilemmas in large language models.” 2025.
6. A. Vaswani et al. “Attention Is All You Need.” NeurIPS 2017.
7. J. Kaplan et al. “Scaling Laws for Neural Language Models.” 2020.